

EMPLOYEE MANAGEMENT SYSTEM

Complete Technical Documentation

Version 1.0.0

Generated: June 18, 2026

TECHNOLOGY STACK

Spring Boot 3.2.5	MongoDB	Thymeleaf	Spring Security
OAuth2 / Google	iTextPDF 7.2.5	Docker	Render

A full-featured, cloud-deployable Employee Management System built with Java, Spring Boot, and MongoDB — supporting Admin & Employee roles, attendance tracking, leave management, salary processing, and more.

Table of Contents

#	Section	Page
1	Project Overview	3
2	Technology Stack	4
3	System Architecture	5
4	Database Design	6
5	Security	7
6	Features	8
7	API Endpoints	9
8	Deployment	10
9	Conclusion	11

1. Project Overview

What is the Employee Management System?

The Employee Management System (EMS) is a full-stack web application designed to digitise and streamline all HR processes within an organisation. It provides a centralised platform where HR administrators and employees can manage day-to-day operational tasks efficiently, securely, and from anywhere in the world via a cloud deployment.

Purpose

Traditional HR workflows — attendance registers, paper-based leave forms, manual salary calculations — are slow and error-prone. EMS replaces these with an automated, role-aware digital system that saves time and reduces administrative overhead.

Project Goals

- Provide a secure, role-based portal for Admins and Employees.
- Automate attendance tracking, leave approvals, and payroll generation.
- Enable real-time communication through announcements and messaging.
- Support Google OAuth2 for frictionless, passwordless login.
- Deploy to the cloud using Docker containers on Render with MongoDB Atlas.
- Generate professional PDF reports and documentation on demand.
- Maintain a clean, responsive UI built with Thymeleaf and custom CSS.

Target Users

- HR Administrators — full system control, approvals, reporting.
- Employees — self-service portal for attendance, leave, tasks, and salary slips.

2. Technology Stack

Spring Boot 3.2.5	<i>An opinionated, production-ready framework built on top of Spring Framework.</i> Rapid development with auto-configuration, embedded Tomcat server, and a rich ecosystem of starters reduce boilerplate and accelerate delivery.
MongoDB	<i>A NoSQL, document-oriented database that stores data in flexible BSON documents.</i> Schema flexibility suits HR data models that evolve over time; horizontal scalability and MongoDB Atlas provide a managed cloud database.
Thymeleaf	<i>A modern server-side Java template engine for rendering HTML views.</i> Native Spring Boot integration, natural templating syntax, and rich attribute expressions keep view logic clean and testable.
Spring Security	<i>A comprehensive authentication and authorisation framework for Spring applications.</i> Provides form login, role-based access control (ADMIN / EMPLOYEE), CSRF protection, and session management out of the box.
OAuth2 / Google Login	<i>An industry-standard authorisation protocol enabling third-party identity providers.</i> Employees can sign in with Google accounts, eliminating password fatigue and improving security through Google's hardened auth infrastructure.
iTextPDF 7.2.5	<i>A powerful Java PDF creation and manipulation library.</i> Enables server-side generation of salary slips, attendance reports, and this documentation without any external processes.
Docker	<i>A containerisation platform that packages applications and their dependencies.</i> Guarantees environment parity between development and production; a single Dockerfile describes the entire runtime.
Render	<i>A fully managed cloud platform for deploying web services and containers.</i>



Zero-downtime deploys, automatic TLS, and simple GitHub integration make production deployments straightforward and cost-effective.

3. System Architecture

EMS follows a classic layered (N-tier) architecture with clear separation of concerns across five layers. Each layer communicates only with its immediate neighbour, keeping the codebase modular and testable.

```
CLIENT BROWSER
(Thymeleaf HTML + CSS + JavaScript)

HTTP/HTTPS

PRESENTATION LAYER
Spring MVC Controllers + REST Endpoints


SERVICE LAYER
Business Logic • Validations • PDF Generation


REPOSITORY LAYER
Spring Data MongoDB (MongoRepository)

BSON / JSON

DATA LAYER
MongoDB Atlas (Cloud Database)
```

Security Cross-Cut

Spring Security sits as a filter chain in front of every Controller, enforcing authentication and role-based authorisation before any business logic executes. OAuth2 token exchange is handled transparently via Spring's OAuth2 client support.

Deployment Architecture

```
GitHub Repository
git push / PR merge

Render.com    Docker Build    Container Deploy

MongoDB Atlas (Cloud DB)

HTTPS Public URL    End Users
```

4. Database Design

All data is persisted in MongoDB Atlas. The system uses 14 collections, each mapped to a Spring Data MongoDB document class.

Collection	Fields
employees	id, employeeId, name, email, password, role, department, designation, phone, address, salary, joinDate, profilePic, active
departments	id, name, description, headName, createdAt
attendance	id, employeeId, date, checkIn, checkOut, status (PRESENT/ABSENT/HALF_DAY), hoursWorked
leaves	id, employeeId, employeeName, leaveType, startDate, endDate, days, reason, status (PENDING/APPROVED/REJECTED), appliedAt, respondedAt
salaries	id, employeeId, month, year, basicSalary, allowances, deductions, netSalary, generatedAt, generatedBy
tasks	id, title, description, assignedTo, assignedBy, dueDate, priority, status (TODO/IN_PROGRESS/DONE), createdAt
announcements	id, title, content, postedBy, postedAt, targetRole
messages	id, senderId, senderName, receiverId, receiverName, content, timestamp, read
notifications	id, userId, message, type, read, createdAt
complaints	id, employeeId, employeeName, subject, description, status (OPEN/RESOLVED), response, submittedAt, resolvedAt
work_from_home	id, employeeId, employeeName, startDate, endDate, reason, status (PENDING/APPROVED/REJECTED), requestedAt
performance	id, employeeId, reviewerName, period, rating (1-5), comments, createdAt
holidays	id, name, date, type (PUBLIC/OPTIONAL), description
users	id, username (email), password (BCrypt), role, enabled, oauthProvider

5. Security

Role-Based Access Control (RBAC)

EMS defines two roles stored in MongoDB: `ROLE_ADMIN` and `ROLE_EMPLOYEE`. Every HTTP request is evaluated by the `SecurityFilterChain` before reaching a controller. Admin-only endpoints (employee CRUD, leave approvals, salary generation) require `ROLE_ADMIN`, while self-service endpoints require `ROLE_ADMIN` or `ROLE_EMPLOYEE`.

Spring Security Configuration

- `BCryptPasswordEncoder` (strength 10) hashes all locally stored passwords.
- `DaoAuthenticationProvider` loads user details from MongoDB via `CustomUserDetailsService`.
- Form login endpoint: `POST /login` — redirects to `/dashboard` on success.
- Failed login redirects to `/login?error=true` with a user-friendly message.
- Logout via `POST /logout` invalidates session, clears `JSESSIONID` cookie.
- CSRF protection is enabled by default for all state-changing requests.

Google OAuth2 Login

Employees may choose 'Sign in with Google' on the login page. The OAuth2 flow:

```
User clicks 'Sign in with Google'
Spring redirects    Google Authorization Server
User consents      Google issues Authorization Code
Spring exchanges code    Access Token + UserInfo
CustomOAuth2UserService    loads/creates user in MongoDB
OAuth2LoginSuccessHandler    sets role, redirects to /dashboard
```

Public Endpoints (No Auth Required)

- `GET /` — Landing / redirect page
- `GET /login` — Login page
- `GET /register` — Registration page
- `GET /project-documentation` — This PDF file
- Static assets: `/css/**`, `/js/**`, `/images/**`

6. Features

Admin Features

Employee CRUD — add, edit, view, delete employees; export CSV.

Department Management — create and manage departments.

Attendance Management — view all attendance records; edit entries.

Leave Management — view leave requests, approve or reject with comments.

Salary Generation — compute and generate monthly payslips per employee.

Task Assignment — create tasks, assign to employees, set priority & deadline.

Announcements — post organisation-wide or role-targeted announcements.

WFH Requests — approve or reject employee Work From Home requests.

Complaints — view and respond to employee complaints.

Performance Reviews — add and delete performance evaluation records.

Holiday Calendar — add and remove public / optional holidays.

Reports — generate PDF reports for attendance, salary, and employees.

Notifications — system-generated alerts for key admin actions.

Employee Features

Dashboard — personalised summary: attendance %, pending leaves, upcoming tasks.

My Attendance — mark daily attendance (check-in / check-out); view history.

Leave Application — submit leave requests; track approval status.

Salary Slips — view and download monthly payslips as PDF.

Task Tracker — view assigned tasks, update status (TODO IN_PROGRESS DONE).

Work From Home — submit and track WFH requests.

Complaints — raise HR complaints and track responses.

Announcements — read all organisation-wide and personal announcements.

Employee Features

Messaging — internal messaging with colleagues and HR.

Performance — view personal performance review history.

Holiday Calendar — view all upcoming holidays.

Profile Management — update personal details and profile picture.

Change Password — securely update login credentials.

7. API Endpoints

EMS exposes Spring MVC routes for both server-rendered pages (Thymeleaf) and JSON REST responses. All authenticated endpoints require a valid session cookie.

Auth & Public

Method	Path	Description
GET	/login	Login page
POST	/login	Form login submit
GET	/register	Registration page
POST	/register	Submit registration
POST	/logout	Logout (invalidates session)
GET	/project-documentation	Download this PDF

Dashboard

Method	Path	Description
GET	/dashboard	Main dashboard (role-aware)

Employees (Admin)

Method	Path	Description
GET	/employees	List all employees
GET	/addEmployee	Add employee form
POST	/saveEmployee	Save new / edited employee
GET	/editEmployee/{id}	Edit employee form
GET	/deleteEmployee/{id}	Delete employee
GET	/downloadEmployees	Export employees as CSV

Attendance

Method	Path	Description
GET	/attendance	View attendance records

Method	Path	Description
POST	/saveAttendance	Admin save attendance
POST	/markMyAttendance	Employee check-in / check-out

Leave

Method	Path	Description
GET	/leave	View leaves
POST	/applyLeave	Submit leave request
GET	/approveLeave/{id}	Approve leave (Admin)
GET	/rejectLeave/{id}	Reject leave (Admin)

Salary

Method	Path	Description
GET	/salary	View salary records
POST	/salary/generate	Generate payslip (Admin)
GET	/salary/download/{id}	Download salary slip PDF

Tasks

Method	Path	Description
GET	/tasks	View tasks
POST	/tasks/assign	Assign task (Admin)
POST	/tasks/updateStatus	Update task status
GET	/tasks/delete/{id}	Delete task (Admin)

Other Modules

Method	Path	Description
*	/departments/**	Department CRUD (Admin)
*	/announcements/**	Announcements
*	/messages/**	Internal messaging
*	/wfh/**	Work From Home requests

Method	Path	Description
*	/complaints/**	Complaints
*	/notifications/**	Notifications
*	/performance/**	Performance reviews
*	/holidays/**	Holiday calendar
*	/reports/**	PDF reports (Admin)
GET	/my-profile	View / edit profile
GET	/change-password	Change password page

8. Deployment

Docker

The application ships with a multi-stage Dockerfile. The build stage compiles the Spring Boot fat-JAR with Maven, and the runtime stage uses a slim JRE 17 image to keep the final image small and secure.

```
# Build stage
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY . .
RUN mvn clean package -DskipTests

# Runtime stage
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Render

Render.com hosts the containerised application as a Web Service:

- Connect GitHub repository to Render — auto-deploy on every push to main.
- Set environment variables (MONGO_URI, GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, etc.) in Render dashboard.
- Render provisions a free TLS certificate and assigns a public HTTPS URL.
- Health-check endpoint ensures zero-downtime rolling deploys.

MongoDB Atlas

MongoDB Atlas provides a fully managed, globally distributed cloud database:

- Create a free M0 cluster in the AWS region closest to your Render service.
- Add the Render service IP (or 0.0.0.0/0 for simplicity) to Atlas IP Access List.
- Set MONGO_URI in Render environment variables using the Atlas connection string.
- Enable Atlas automatic backups and monitoring for production resilience.

Environment Variables

Variable	Description
MONGO_URI	Full MongoDB Atlas connection string

Variable	Description
GOOGLE_CLIENT_ID	Google OAuth2 client ID
GOOGLE_CLIENT_SECRET	Google OAuth2 client secret
SERVER_PORT	App port (default 8080)
SPRING_PROFILES_ACTIVE	Active Spring profile (prod)

9. Conclusion

The Employee Management System is a production-ready, cloud-native application that demonstrates the power of the Spring ecosystem combined with MongoDB's flexible document model. It covers the full lifecycle of an HR workflow: from hiring an employee and managing their day-to-day activities, to processing payroll and tracking performance.

The system's layered architecture ensures clean separation of concerns, making it straightforward to extend — adding new modules, integrating third-party services, or migrating to microservices — without disrupting existing functionality.

Security is treated as a first-class concern through role-based access control, BCrypt password hashing, CSRF protection, and Google OAuth2, ensuring that sensitive HR data is protected at every layer.

Docker containerisation and Render deployment mean the application is portable and can be shipped to any cloud provider with minimal configuration changes, while MongoDB Atlas handles all database infrastructure concerns.

Employee Management System — Technical Documentation

Built with Spring Boot 3.2.5 • MongoDB • Thymeleaf • Docker • Render

Generated: June 18, 2026